Java.lang

java.lang is automatically imported by Java.

import java.lang.*;

Class / Interface	Purpose	Simple Example
Object	Root class of Java's class hierarchy	Object o = new Object();
String	Represents text	String s = "Hello";
StringBuilder	Creates/modifies mutable strings	StringBuilder sb = new StringBuilder("Hi");
StringBuffer	Mutable string, synchronized	StringBuffer sb = new StringBuffer("Hi");
System	Provides system-related facilities	System.out.println("Hello");
Math	Mathematical operations	Math.sqrt(25)
Integer	Wrapper class for int	Integer x = 10;
Double	Wrapper class for double	Double d = 10.5;
Character	Wrapper class for char	Character.isDigit('5')
Boolean	Wrapper class for boolean	Boolean b = true;
Long	Wrapper class for long	Long n = 100L;
Float	Wrapper class for float	Float f = 10.5f;
Byte	Wrapper class for byte	Byte b = 10;
Short	Wrapper class for short	Short s = 20;
Number	Superclass of numeric wrapper classes	Number n = 10;
Class	Represents a class at runtime	String.class
Thread	Creates and controls threads	Thread t = new Thread();
Runnable	Represents a task that can run in a thread	Runnable r = () -> {};
Exception	Base class for many exceptions	throw new Exception();
RuntimeException	Base class for unchecked	throw new RuntimeException();

	exceptions	
Error	Represents serious JVM/system errors	OutOfMemoryError
Enum	Base class for enum types	enum Day { MON, TUE }
Process	Represents an operating-system process	Process p;
ProcessBuilder	Creates operating-system processes	new ProcessBuilder()

Object — Root Class

Object is the **parent class of every Java class**.

```
class Student {
}
```

Internally, it is equivalent to:

```
class Student extends Object {
}
```

Method	Purpose	Example
toString()	Returns string representation	s.toString()
equals()	Compares objects	a.equals(b)
hashCode()	Returns hash code	s.hashCode()
getClass()	Returns runtime class information	s.getClass()
clone()	Creates a copy under suitable conditions	s.clone()
wait()	Makes thread wait	obj.wait()
notify()	Wakes a waiting thread	obj.notify()
notifyAll()	Wakes all waiting threads	obj.notifyAll()

```
Integer s = 42;
System.out.println(s.toString()); // Output: "42"
```

```
String s = "hello";
int hash = s.hashCode(); // Calculates the hash code for the String "hello"
System.out.println(hash); // Output: 99162322
```

```
String a = new String("hello");
String b = new String("hello");

System.out.println(a == b);      // false (different memory locations)
System.out.println(a.equals(b)); // true  (same character content)
```

```
Object s = "Hello, World!";
System.out.println(s.getClass());
// Output: class java.lang.String

System.out.println(s.getClass().getName());
// Output: java.lang.String
```

```
int[] s = {1, 2, 3};
int[] copy = s.clone();

System.out.println(Arrays.toString(copy)); // Output: [1, 2, 3]
System.out.println(s == copy);             // Output: false (they are distinct array objects)
Arrays.equals(s, copy) → true (element comparison)
```

String: (Immutable)

A String object is **immutable**, meaning its contents cannot be changed after creation.

```
String s = "Hello";
s = s + " World";
System.out.println(s);
Hello World
// Here, the original "Hello" object is not changed. A new String object is created.
```

Method	Purpose	Example	Output
length()	Number of characters	"Hello".length()	5
charAt()	Gets character at index	"Hello".charAt(0)	H
substring()	Extracts part of string	"Hello".substring(1,4)	ell
toUpperCase()	Converts to uppercase	"hello".toUpperCase()	HELLO
toLowerCase()	Converts to lowercase	"HELLO".toLowerCase()	hello
equals()	Compares strings	"abc".equals("abc")	true
contains()	Checks for text	"Hello".contains("ell")	true
indexOf()	Finds position	"Hello".indexOf('l')	2
replace()	Replaces characters/text	"cat".replace('c','b')	bat
trim()	Removes leading/trailing spaces	" Hi ".trim()	Hi

StringBuilder: (Mutable)

StringBuilder is used when we want to **modify a string repeatedly**.

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");
System.out.println(sb);
Hello Java
// Here, the same StringBuilder object is modified.
```

Method	Example	Result
append()	sb.append("Java")	Adds at end
insert()	sb.insert(0, "Hi ")	Inserts text
delete()	sb.delete(0, 3)	Deletes characters
reverse()	sb.reverse()	Reverses
replace()	sb.replace(0, 3, "Hey")	Replaces
length()	sb.length()	Length

StringBuffer: (Mutable + Thread Safe)

StringBuffer is similar to StringBuilder.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" Java");
System.out.println(sb);
//It is similar to StringBuilder, but its methods are synchronized, making it suitable when multiple threads may access the same object.
```

StringBuilder vs StringBuffer

Feature	StringBuilder	StringBuffer
Mutable	Yes	Yes
Thread-safe	No	Yes
Synchronization	No	Yes
Usually faster	Yes	Relatively slower
Package	java.lang	java.lang

System

System provides several useful system-level facilities

```
System.out.println("Hello");
```

Member	Purpose
System.out	Standard output
System.in	Standard input
System.err	Standard error
currentTimeMillis()	Current time in milliseconds
nanoTime()	High-resolution time measurement
exit()	Terminates the program
arraycopy()	Copies arrays
gc()	Requests garbage collection
getProperty()	Gets system property

```
System.out.println("Regular message");
System.err.println("Error message");
String value = System.getProperty("property.key");
//user.name, user.dir, os.name, java.version etc.,
```

Math

Math provides mathematical functions.

Method	Example	Output
sqrt()	Math.sqrt(25)	5.0
pow()	Math.pow(2,3)	8.0
abs()	Math.abs(-10)	10
max()	Math.max(10,20)	20
min()	Math.min(10,20)	10
round()	Math.round(4.6)	5
ceil()	Math.ceil(4.2)	5.0
floor()	Math.floor(4.8)	4.0
random()	Math.random()	Random value

Wrapper Classes

Java has primitive data types:

Primitive	Wrapper
byte	Byte

short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Integer

Integer is the wrapper class for int

Integer x = 100;

Method	Example	Result
parseInt()	Integer.parseInt("123")	123
valueOf()	Integer.valueOf("123")	Integer object
toString()	Integer.toString(100)	"100"
compare()	Integer.compare(10,20)	-1
max()	Integer.max(10,20)	20
min()	Integer.min(10,20)	10

Double

Wrapper class for double

Double d = 25.5;

```
double x = Double.parseDouble("25.5");
System.out.println(x + 10);
35.5
```

Character

Character provides methods for working with individual characters.

Method	**Example**	**Result**
isDigit()	Character.isDigit('5')	true
isLetter()	Character.isLetter('A')	true
isUpperCase()	Character.isUpperCase('A')	true
isLowerCase()	Character.isLowerCase('a')	true

toUpperCase()	Character.toUpperCase('a')	A
toLowerCase()	Character.toLowerCase('A')	a
isWhitespace()	Character.isWhitespace(' ')	true

Boolean

Wrapper class for boolean.

Boolean b = true;

```
boolean x = Boolean.parseBoolean("true");
System.out.println(x);
True
```

Number

Number is an abstract class that acts as the superclass for numeric wrapper classes.

Number n = 100;

```
public class Main {
    public static void main(String[] args) {
        Number n = 25.75;
        System.out.println(n.intValue());
        System.out.println(n.longValue());
        System.out.println(n.floatValue());
        System.out.println(n.doubleValue());
    }
}
25
25
25.75
25.75
```

Class

Class provides information about a class at runtime.

```
String s = "Hello";
Class c = s.getClass(); // Which class does this object belong to?
System.out.println(c.getName()); // What is the name of that class?
```

Thread

Thread is used for **multithreaded programming**.

```
class Demo extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
    public static void main(String[] args) {
        Demo d = new Demo();
        d.start();
    }
}
```

Method	Purpose
start()	Starts a thread
run()	Contains thread's task
sleep()	Pauses execution
join()	Waits for another thread
getName()	Gets thread name
setName()	Sets thread name
getPriority()	Gets priority
setPriority()	Sets priority
currentThread()	Gets currently executing thread
isAlive()	Checks whether thread is alive

Runnable

Runnable is an interface used to define a task that can be executed by a thread.

```
class Demo implements Runnable {
    public void run() {
        System.out.println("Running");
    }
    public static void main(String[] args) {
        Demo d = new Demo();
```

```
        Thread t = new Thread(d);
        t.start();
    }
}
```

Exception

An exception is an event that occurs during program execution and disrupts the normal flow of the program.

Compile Time Exception (Checked Exception)

```
import java.io.*;
public class Main {
    public static void main(String[] args) {
        try {
            FileReader f = new FileReader("test.txt");
            f.close();
        }
        catch (IOException e) {
            System.out.println("Input/Output error occurred");
        }
    }
}
```

IOException	An input/output operation fails
FileNotFoundException	FileNotFoundException
SQLException	SQLException
ClassNotFoundException	ClassNotFoundException
InterruptedException	InterruptedException

RuntimeException

RuntimeException is the superclass of many **unchecked exceptions**.

```
try {
    int x = 10 / 0;
}
catch (Exception e) {
    System.out.println("Cannot divide by zero");
```

```
}
```

Cannot divide by zero

Exception	Example	Reason
ArithmeticException	int x = 10 / 0;	Division by zero
NullPointerException	String s = null; s.length();	Using a null reference
ArrayIndexOutOfBoundsException	int[] a = {10, 20}; System.out.println(a[5]);	Invalid array index
NumberFormatException	int x = Integer.parseInt("abc");	Invalid number conversion

Error

Error represents serious problems generally associated with the JVM or runtime environment.

Examples include:

OutOfMemoryError

StackOverflowError

NoClassDefFoundError

Example for OutOfMemoryError

```
class Main {
    public static void main(String[] args) {
        int[] arr = new int[1000000000];
    }
}
```

Example for StackOverflowError

```
class Demo {
    static void test() {
        test();
    }
    public static void main(String[] args) {
        test();
    }
}
```

The method repeatedly calls itself until the stack cannot accommodate more calls.

Enum

Enum is the base class for all Java enum types.

```
enum Day {
```

```
    MONDAY, TUESDAY, WEDNESDAY
}
```

Process and ProcessBuilder

These classes are used to work with operating-system processes.

ProcessBuilder can be used to create a process.

Open Notepad

```
public class Main {
    public static void main(String[] args) throws Exception {
        ProcessBuilder pb = new ProcessBuilder("notepad");
        Process p = pb.start();
        System.out.println("Notepad started");
    }
}
```

Open Calculator

```
public class Main {
    public static void main(String[] args) throws Exception {
        ProcessBuilder pb = new ProcessBuilder("calc");
        pb.start();
    }
}
```

Open Paint

```
public class Main {
    public static void main(String[] args) throws Exception {

        ProcessBuilder pb = new ProcessBuilder("mspaint");
        pb.start();
    }
}
```

Open a website

```
public class Main {
    public static void main(String[] args) throws Exception {

        ProcessBuilder pb =
            new ProcessBuilder("cmd", "/c", "start", "https://www.google.com");

        pb.start();
    }
}
```